

1. Remove duplicates from an array without using HashSet.

Simple Explanation: The most straightforward way without a HashSet is to first **sort the array**. Once sorted, identical elements will be next to each other. We then use a two-pointer approach to build a new, unique array or modify the existing one in-place, copying only the unique elements.

```
public class DuplicateRemover {
    public static int removeDuplicatesSorted(int[] arr) {
        if (arr == null || arr.length <= 1) {
            return arr.length;
        }

        // 1. Sort the array first
        java.util.Arrays.sort(arr);

        int uniqueCount = 1;
        int j = 0; // Pointer for the next unique element position

        // 2. Iterate and copy unique elements
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] != arr[j]) {
                j++;
                arr[j] = arr[i]; // Overwrite the next unique spot
                uniqueCount++;
            }
        }
        // uniqueCount is the length of the new array; the unique elements
        // are in arr[0...uniqueCount-1]
        return uniqueCount;
    }
}
```

2. Find missing numbers from an array of size n with values 1–n.

Simple Explanation: We can find the missing number by comparing the expected sum of all numbers from 1 to n with the actual sum of the numbers present in the array. The difference between these two sums will be the missing number.

```
public class MissingNumberFinder {
    public static int findMissing(int[] arr, int n) {
        // Formula for the sum of numbers from 1 to n: n * (n + 1) / 2
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;

        for (int num : arr) {
            actualSum += num;
        }
    }
}
```

```
        // The difference is the missing number
        return expectedSum - actualSum;
    }
}
```

3. Two-sum : return indices of two numbers adding to target.

Simple Explanation: We use a **HashMap** to store numbers we have already seen along with their indices. For each number in the array, we calculate the complement needed to reach the target. We check if the complement exists in the map; if it does, we found the pair and return both indices.

```
import java.util.HashMap;
import java.util.Map;

public class TwoSum {
    public static int[] findTwoSum(int[] nums, int target) {
        Map<Integer, Integer> numMap = new HashMap<>(); // Value -> Index

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];

            // Check if the required complement is already in the map
            if (numMap.containsKey(complement)) {
                // Pair found: return the current index and the
                // complement's index
                return new int[]{numMap.get(complement), i};
            }

            // Store the current number and its index for future checks
            numMap.put(nums[i], i);
        }

        // If no solution found
        throw new IllegalArgumentException("No two sum solution");
    }
}
```

4. Find the intersection of two arrays.

Simple Explanation: The intersection consists of elements present in both arrays. We use a **HashSet** to store all elements from the first array. Then, we loop through the second array, checking if the element exists in the set. If it exists, it's part of the intersection.

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class ArrayIntersection {
    public static List<Integer> findIntersection(int[] arr1, int[] arr2)
    {
        Set<Integer> set1 = new HashSet<>();
        List<Integer> intersection = new ArrayList<>();

        // Add all elements from the first array to a set
        for (int num : arr1) {
            set1.add(num);
        }

        // Check if elements from the second array are in the set
        for (int num : arr2) {
            if (set1.contains(num)) {
                intersection.add(num);
                set1.remove(num); // Remove from set1 to handle duplicates
            }
        }
        return intersection;
    }
}
```

5. Find the union of two arrays.

Simple Explanation: The union consists of all unique elements from both arrays. We simply use a single **HashSet** and add every element from the first array and then every element from the second array into the set. The set automatically handles duplicates, ensuring only unique elements remain.

```
import java.util.HashSet;
import java.util.Set;

public class ArrayUnion {
    public static Set<Integer> findUnion(int[] arr1, int[] arr2) {
        Set<Integer> unionSet = new HashSet<>();

        // Add all elements from both arrays
        for (int num : arr1) {
            unionSet.add(num);
        }
        for (int num : arr2) {
            unionSet.add(num);
        }
    }
}
```

```
        return unionSet;
    }
}
```

6. Rotate an array by k positions.

Simple Explanation: To rotate an array (right rotation is simpler here) by k positions, we can reverse the array in three steps: 1) Reverse the first $n-k$ elements. 2) Reverse the last k elements. 3) Reverse the entire array.

```
public class ArrayRotator {
    public static void rotate(int[] nums, int k) {
        if (nums == null || k < 0) {
            return;
        }

        k = k % nums.length; // Handle rotations larger than array size
        int n = nums.length;

        // 1. Reverse the whole array
        reverse(nums, 0, n - 1);

        // 2. Reverse the first k elements
        reverse(nums, 0, k - 1);

        // 3. Reverse the remaining n - k elements
        reverse(nums, k, n - 1);
    }

    private static void reverse(int[] nums, int start, int end) {
        while (start < end) {
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

7. Find the maximum product of two integers in an array.

Simple Explanation: The maximum product will always come from either the two largest positive numbers OR the two smallest negative numbers (since multiplying two negatives results in a positive). We find the four extremes (two largest and two smallest) in a single pass and compare the resulting products.

```
public class MaxProductFinder {
    public static int findMaxProduct(int[] nums) {
        if (nums == null || nums.length < 2) {
            throw new IllegalArgumentException("Array must have at least
two elements.");
        }

        // Find the four extreme values in one pass
        int max1 = Integer.MIN_VALUE, max2 = Integer.MIN_VALUE;
        int min1 = Integer.MAX_VALUE, min2 = Integer.MAX_VALUE;

        for (int n : nums) {
            if (n > max1) {
                max2 = max1; max1 = n;
            } else if (n > max2) {
                max2 = n;
            }
            if (n < min1) {
                min2 = min1; min1 = n;
            } else if (n < min2) {
                min2 = n;
            }
        }

        // Compare the product of the two largest vs. the product of the
two smallest
        return Math.max(max1 * max2, min1 * min2);
    }
}
```

8. Find the majority element in an array. (Boyer–Moore or counting)

Simple Explanation (Boyer-Moore Voting Algorithm): The majority element appears more than $n/2$ times. We track a candidate element and a counter. If the current element matches the candidate, we increase the counter; otherwise, we decrease it. If the counter hits zero, we select the next element as the new candidate.

```
public class MajorityElementFinder {
    public static int findMajority(int[] nums) {
        int candidate = 0;
        int count = 0;

        // Phase 1: Find the candidate element
        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            if (num == candidate) {
                count++;
            } else {
                count--;
            }
        }
    }
}
```

```

        }
    }

    // (Optional Phase 2: Verification, not needed if guaranteed
    majority exists)
    // Simply return the candidate under the assumption that a
    majority element exists
    return candidate;
}
}

```

9. Kadane's Algorithm : maximum sum subarray.

Simple Explanation: Kadane's algorithm efficiently finds the subarray with the largest possible sum. We iterate through the array, tracking two variables: `currentMax` (the maximum sum ending at the current position) and `globalMax` (the maximum sum found so far). If `currentMax` ever drops below zero, we reset it to zero to start a new, potentially positive subarray.

```

public class KadanAlgorithm {
    public static int maxSubArraySum(int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        int max_so_far = nums[0]; // Global maximum found so far
        int current_max = nums[0]; // Maximum sum ending at the current
        position

        for (int i = 1; i < nums.length; i++) {
            // Either continue the current subarray or start a new one
            current_max = Math.max(nums[i], current_max + nums[i]);

            // Update the global maximum if current_max is better
            max_so_far = Math.max(max_so_far, current_max);
        }
        return max_so_far;
    }
}

```

10. Find all pairs in array whose sum equals a given number.

Simple Explanation: We use a **HashSet** to store the numbers we have already seen. For each number in the array, we calculate the complement needed to reach the target sum. We check if this complement exists in the set; if it does, we found a pair.

```
import java.util.HashSet;
import java.util.Set;

public class PairFinder {
    public static void findAllPairs(int[] nums, int target) {
        Set<Integer> seen = new HashSet<>();

        for (int num : nums) {
            int complement = target - num;

            // Check if the complement was already seen
            if (seen.contains(complement)) {
                System.out.println("Pair found: (" + num + ", " +
complement + ")");
            }

            // Add the current number to the set of seen numbers
            seen.add(num);
        }
    }
}
```

11. Remove all instances of a value in-place from array.

Simple Explanation: We use a two-pointer technique to solve this **in-place**. We use one pointer to iterate and another pointer (`write_index`) to track where the next valid element should be placed. If the current element is **not** the value to remove, we copy it to the `write_index` position and advance the pointer.

```
public class ValueRemover {
    public static int removeValueInPlace(int[] nums, int val) {
        if (nums == null) {
            return 0;
        }

        int write_index = 0; // Tracks where to write the next valid
element

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                // Copy the valid element to the front
                nums[write_index] = nums[i];
                write_index++;
            }
        }
        // write_index is the new length of the array after removal
        return write_index;
    }
}
```

12. Count frequency of elements in an array.

Simple Explanation: We use a **HashMap** where the key is the element, and the value is its count. We loop through the array; for each element, we check if it's already in the map. If it is, we increase its count; otherwise, we add it with a starting count of 1.

```
import java.util.HashMap;
import java.util.Map;

public class FrequencyCounter {
    public static Map<Integer, Integer> countFrequency(int[] arr) {
        Map<Integer, Integer> frequencyMap = new HashMap<>();

        for (int num : arr) {
            // Use getOrDefault to simplify incrementing the count
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        return frequencyMap;
    }
}
```

13. Merge two sorted arrays.

Simple Explanation: We create a new array large enough to hold both inputs. We use three pointers: one for the first array, one for the second array, and one for the new merged array. We compare the elements pointed to by the first two pointers, copying the smaller element into the merged array and advancing that pointer.

```
public class ArrayMerger {
    public static int[] mergeSorted(int[] arr1, int[] arr2) {
        int n1 = arr1.length;
        int n2 = arr2.length;
        int[] merged = new int[n1 + n2];

        int i = 0, j = 0, k = 0; // Pointers for arr1, arr2, and merged

        // While there are elements in both arrays
        while (i < n1 && j < n2) {
            if (arr1[i] <= arr2[j]) {
                merged[k++] = arr1[i++];
            } else {
                merged[k++] = arr2[j++];
            }
        }

        // Copy remaining elements from arr1 (if any)
        while (i < n1) {
            merged[k++] = arr1[i++];
        }

        // Copy remaining elements from arr2 (if any)
    }
}
```



```
        while (j < n2) {
            merged[k++] = arr2[j++];
        }

        return merged;
    }
}
```

14. Find element appearing only once where others appear twice.

Simple Explanation: We use the **XOR (Exclusive OR)** bitwise operator. The key property of XOR is that $A \oplus A = 0$ and $A \oplus 0 = A$. By XORing all elements together, all pairs cancel each other out to zero, and the single unique element remains as the final result.

```
public class UniqueElementFinder {
    public static int findSingle(int[] nums) {
        int result = 0; // The identity element for XOR is 0

        // XOR all elements in the array
        for (int num : nums) {
            result = result ^ num;
        }

        // The result holds the unique number
        return result;
    }
}
```

15. Find the first repeating element in an array.

Simple Explanation: We use a **HashSet** to track seen elements. We iterate through the array *backward* (from right to left). This way, the first element we find that is already in the set will be the *first repeating* element when reading the array from the beginning.

```
import java.util.HashSet;
import java.util.Set;

public class FirstRepeatingFinder {
    public static int findFirstRepeating(int[] nums) {
        if (nums == null || nums.length == 0) {
            return -1;
        }

        Set<Integer> seen = new HashSet<>();
        int firstRepeating = -1;

        // Iterate backward
        for (int i = nums.length - 1; i >= 0; i--) {
            if (seen.contains(nums[i])) {

```

```
        firstRepeating = nums[i]; // Store this as the latest  
repeating element  
    }  
    seen.add(nums[i]);  
}  
return firstRepeating;  
}  
}
```

JAVA Interview Buddy